Programmatic and semantic zoom (Chapter 14)

This section contains additional information that is not present in the book.

Zoom interpolation

Zooming uses the `d3.interpolateZoom` interpolator for smooth transitions during double-click and programmatic zooming, making objects change size during a translation even if not scaled. You can eliminate this effect by replacing the default interpolator with the `interpolate()` method.

For example, this replaces it with a linear interpolator (`ZoomProg/10-interpolator.html`):

```
const zoom = d3.zoom().interpolate(d3.interpolate);
```

The current interpolator's curvature can be adjusted using its `rho()` method. Default is `Math.sqrt(2)`. Values closer to 0 are more linear. The same effect above could be obtained with:

```
const zoom = d3.zoom().interpolate(d3.interpolateZoom.rho(0));
```

You can also use `d3.interpolateZoom` to create smooth animations between views. The following example demonstrates a viewport transitioning smoothly between two transformed views, passing through an intermediate untransformed view, as shown in *Figure 1*.

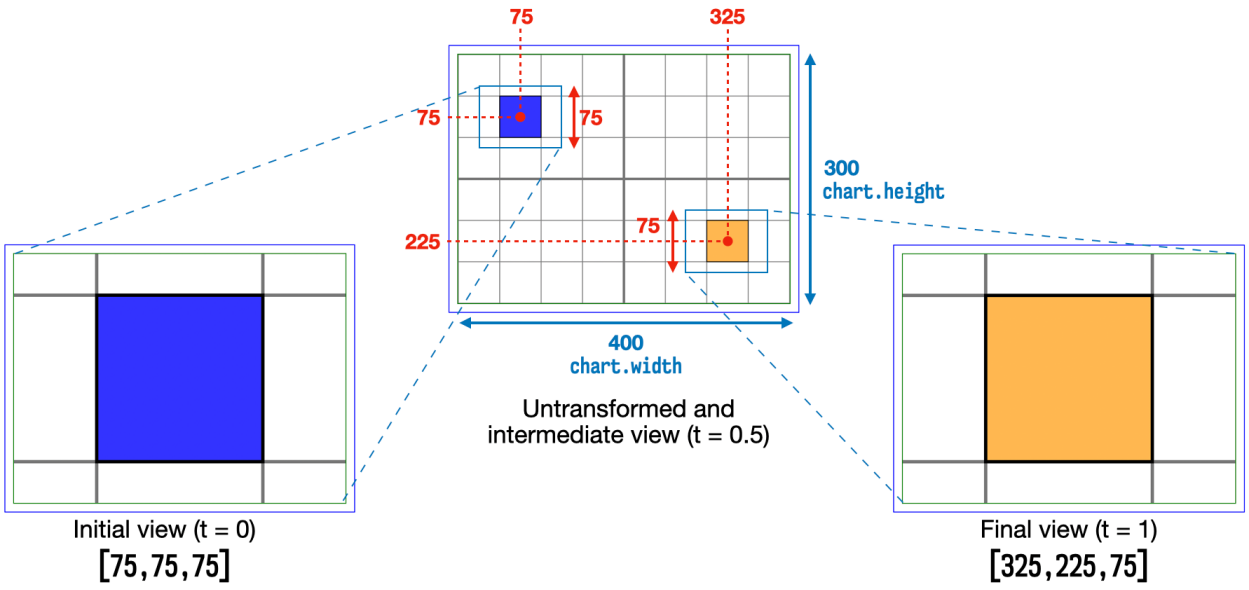

Figure 1 – Computing a zoom transition. Code: ZoomProg/11-interpolateZoom.html.

The interpolator is set up with initial and final views, each defined by a 3-element array containing center coordinates and minimum dimension (width or height):

```
const interpolator = d3.interpolateZoom([75,75,75], [325,225,75]);
```

During a transition, the view will gradually move from `interpolator(0)`, which is `[75,75,75]` to `interpolator(1)`, which is `[325,225,75]`. The transforms that obtain each intermediate view need to be computed for each transition step *t*. This is done in the following function:

```
function transform(t) {
    const view = interpolator(t);
    const k = Math.min(chart.width, chart.height) / view[2];
    const [x,y] = [chart.width/2 - view[0] * k, chart.height/2 - view[1] * k];
    return d3.zoomIdentity.translate(x,y).scale(k);
}
```

The function computes the view for the interpolator's step (`t`), calculates the scale factor `k` by dividing the chart's width or height by the view's width, and finds `[x,y]` coordinates by subtracting the chart's center from the view's center. It then returns a zoom transform using these results.

Before the animation begins, the initial view obtained via `transform(0)` is applied to the container. The transition initiates and smoothly moves the view from the blue square to the orange one.

```
d3.select(".container")
    .attr("transform", transform(0))          // apply transform to show initial view
    .transition().delay(500).duration(interpolator.duration*2)
                            .attrTween("transform", () => t => transform(t));
```

To see this animation in action, run `ZoomProg/11-interpolateZoom.html`.

Semantic zoom: applying and inverting transforms

This section contains extra details and code examples that were removed from the corresponding section in the book, which just briefly discusses this topic. The functions listed in *Table 1* are covered in this section.

Method	Description
`apply([x1,y1])`	Returns a point `[x2,y2]` that is a relative transformation of the provided point `[x1,y1]`, computed as `[x1*k + x,y1*k + y]`, where {*k, x , y*} is the current transform.
`applyX(x1)` `applyY(y1)`	Returns a number which is a relative transformation of the given *x1* or *y1* coordinate, calculated as *x1*k + x or y1*k + y* where {*k, x , y*} is the current transform.

Method	Description
`invert([x1,y1])`	Returns a point $[x2, y2]$ that is the inverse transformation of the provided point $[x1, y1]$, computed as $[(x1 - x)/k, (y1 - y)/k]$, where $\{k, x, y\}$ is the current transform.
`invertX(x1)` `invertY(y1)`	Returns a number which is the inverse transformation of the given *x1* or *y1* coordinate, calculated as $(x1 - x)/k$, or $(y1 - y)/k$, where $\{k, x, y\}$ is the current transform.

Table 1 – Zoom transform methods that transform individual points and coordinates.

The `apply()`, `applyX()` and `applyY()` methods work by multiplying each coordinate value by the scale factor and then adding the result to the transform's current value. While using `rescaleX()` and `rescaleY()` may be simpler, they alter the scale used and may have an impact in performance in large datasets. This alternative performs pixel computations in screen space and allows you to selectively apply transformations when necessary.

In the example used in the *Implementing semantic scaling* section, from the book, instead of labelling each dot, let's reveal the information with a tooltip. The tooltip is a `<text>` element appended to the fixed `context` container. Since it won't be zoomed, its font-size and relative placement (12 pixels above the dot) are fixed (see the code in `ZoomSemantic/8-apply.html`):

```
d3.select(".context").append("text").attr("class", "tooltip")
                     .attr("y", -12).style("font-size", 10);
```

The tooltip is activated when the user hovers over a circle. The following code updates it with the point's coordinates (stored in `d`), makes it visible, and moves it to the circle's position:

```
d3.select(".container").selectAll("circle")
    .on("pointerover", function(evt, d) {
        d3.select(".tooltip").text(`(${d[0]},${d[1]})`).style("opacity", 1);
          .attr("transform", `translate(${d})`); // This is not zoomed!
})
```

This works fine while zoom is not applied. After zooming, however, the tooltips no longer appear near the circles when the mouse hovers over them (*Figure 2*).

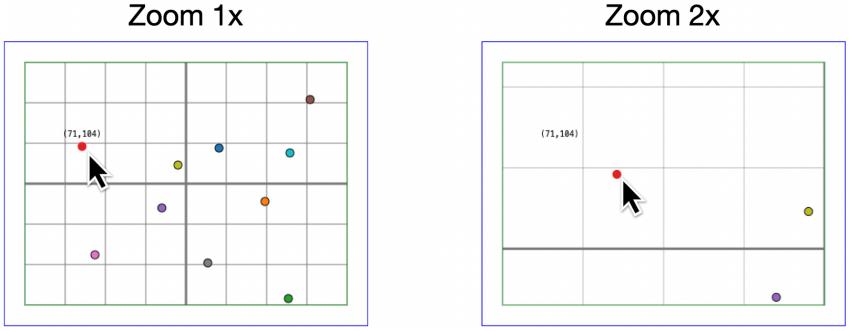

Figure 2 – After zooming, tooltip is no longer near the circle. Code: ZoomSemantic/8-apply.html.

To keep tooltips near the circle, you must apply the current zoom transform to the tooltip's coordinates:

```
.attr("transform", `translate(${currentTransform.apply(d)})`); // Apply zoom transform
```

The `invert()`, `invertX()` and `invertY()` methods perform the reverse operation and are used to obtain untransformed coordinates.

Let's continue with our example and add the ability to create new data points by clicking anywhere in the chart. This will add a new coordinate pair to the `dots` array:

```
d3.select(".chart")
  .on("click", function(evt) {
      dots.push(d3.pointer(evt));
      update();            // updates circles with new data
  });
```

The `update()` function (see full code in `ZoomSemantic/9-invert.html`) uses a join to update the circles on the screen with the new data, adding new circles as the `dots` array grows.

The stored coordinates are relative to the container that contains the circles. When you zoom in and out, the stored data refers to the circles' original positions, but their screen coordinates change. Clicking on the transformed screen will provide incorrect data points and render the circles in the wrong place (see *Figure 3*).

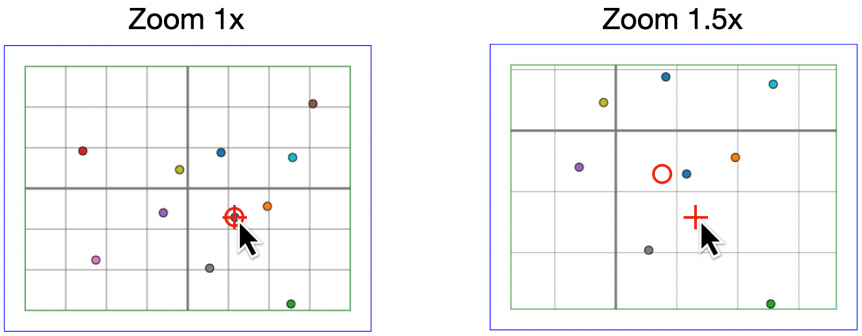

Figure 3 – After zoom, circle is not added at mouse position. Code: ZoomSemantic/9-invert.html.

The solution is to call `transform.invert()` to invert the pointer's coordinates so that the new data is stored in the same dimension as the existing data:

```
dots.push(currentTransform.invert(d3.pointer(evt)));
```

Now new circles are added at mouse position.

If you are using scales, two inversions are required to obtain the data value bound to the original screen coordinates. With *transform*`.invert()` you obtain the untransformed screen coordinates, and the original unscaled data is obtained by passing the result to *scale*`.invert()`:

```
.on("click", function(evt) {
    const [screenX,screenY] = currentTransform.invert(d3.pointer(evt));
    const [x,y] = [scale.x.invert(screenX), scale.y.invert(screenY)];
    dots.push([x,y]);
    update();
});
```

You can run this example in ZoomSemantic/10-invert-scales.html.

In this example, however, instead of inverting pointer coordinates, we could have simply used rescaling methods to adjust the domains of each scale to the transformed space (ZoomSemantic/11-rescale.html):

```
.on("click", function(evt) {
    const [screenX, screenY] = d3.pointer(evt);
    const scaleX = currentTransform.rescaleX(scale.x);
    const scaleY = currentTransform.rescaleY(scale.y);
    dots.push([scaleX.invert(screenX), scaleY.invert(screenY)]);
    update();
});
```